

Assignment 1: Pthread/OpenMP Parallelization

CSIE5213 Parallel Programming

Due: Thu, 2026/10/1 23:59

[UPDATE] **2026-09-23**: Added HW 1-3 (Section 4).

1 Problem Description

You will parallelize:

- an adaptive denoising filter (HW 1-1, Section 2),
- a feature detection & matching pipeline in the style of SIFT (HW 1-2, Section 3), and
- a heat diffusion simulation (HW 1-3, Section 4).

Optimize each of them with multithreading, using either **pthread** or **OpenMP**. You are **not allowed** to use any other multithreading library, including but not limited to `std::thread`. You may modify the code, as long as you do not change the underlying algorithm's computational complexity.

1.1 Using the Cluster

SEIREN is a shared machine lent to this course. Your account on it is for **this homework only**.

The whole class shares the machine, so Slurm limits each student to:

- **1** running job and **2** queued jobs at a time.
- Per job: **8** cores (`-n` / `-c`), **16 GB** of memory, **20** minutes of wall time.
- **10** minutes between two runs of the judge (Section 7.2), counted across all problems.

Request only the resources the judge uses, and cancel any job you no longer need; other students are waiting for the same cores.

In addition, you may **not**:

- run your code outside Slurm: always launch it with `srun` or `sbatch`, and use the login node only for editing and compiling;
- flood SEIREN with jobs or requests, in any form of denial-of-service attack;
- scan, probe or attack SEIREN, or any other machine on this network or anywhere else;
- mine cryptocurrency, or run anything else whose purpose is to burn CPU time for its own sake;
- run a web server, a game server, a bot, or any other service for people outside this course to connect to;
- crawl or scrape websites;
- use it as a jump host, proxy or relay to reach somewhere you could not reach directly;
- store or share copyrighted material you have no right to;

- do anything unlawful, or anything that breaks the computing regulations of the department that owns the machine.

Breaking any of these rules costs **5 points of your term grade** or more, and may fail you in the course. Every job runs under your own account, and Slurm logs what each job requested, which nodes it ran on and for how long, so violations are easy to trace.

If you are unsure whether something is allowed, ask on NTU COOL before running it.

1.2 Copying the Example Sequential Code

We provide example sequential code (`hw1-X.cpp` for HW 1-X) along with other necessary files. Copy the entire folder containing the code to your home directory:

```
cp -r /home/share/pp26/hw1 ~/hw1
```

The public test cases are not part of the copy. They are stored in `/home/share/pp26/cases`, and the `cases` entry in the folder is a symbolic link to that directory, so the copy itself is small. Paths such as `cases/hw1-1/t01.in.png` in this document are relative to your copy.

1.3 Compiling the Code

Before the commands below, run:

```
module load gcc
```

We compile your code with:

```
g++ -std=c++11 -O3 -lpng -lpthread -fopenmp hw1-1.cpp -o hw1-1
g++ -std=c++11 -O3 -lpng -lpthread -fopenmp hw1-2.cpp -o hw1-2
g++ -std=c++11 -O3 -lpthread -fopenmp hw1-3.cpp -o hw1-3
```

We will use `make` to build your code. The default `Makefile` for this homework is in the folder you just copied, and builds whichever of `hw1-1.cpp` / `hw1-2.cpp` / `hw1-3.cpp` you submitted. **If you want to change compilation flags, please include the `Makefile` in your submission.**

1.4 Evaluation Settings for Code Submission

We evaluate your code with a command equivalent to:

```
srun -c ${threads} ./hw1-1 ${input_path} ${output_path}
srun -c ${threads} ./hw1-2 ${imageA_path} ${imageB_path} ${output_path}
srun -c ${threads} ./hw1-3 ${N} ${T} ${seed} ${theta} ${output_path}
```

where `${threads}` is the number of threads, and the remaining arguments are the paths described in each problem's I/O format below. The maximum number of CPU cores (`${threads}`) is **8** for all three problems, but individual test cases may run with fewer. Use `sched_getaffinity` and `CPU_COUNT` to determine how many cores are available to your program, and create that many threads.

2 HW 1-1: Adaptive Filtering

HW 1-1 denoises an image while preserving detail by choosing the averaging window size of each pixel from its local brightness. The program computes the luminance $L = 0.299R + 0.587G + 0.114B$ of each input pixel and assigns the pixel an 11×11 window where $L > 128$ and a 5×5 window elsewhere. Each output channel is the mean of the corresponding input channel over that window, truncated to an integer; for a $k \times k$ window centered on the pixel,

$$c'_{x,y} = \left\lfloor \frac{1}{k^2} \sum_{|i|, |j| \leq \lfloor k/2 \rfloor} c_{\text{clamp}(x+i), \text{clamp}(y+j)} \right\rfloor,$$

where clamp replaces a coordinate outside the image with the nearest border coordinate. A window at the border therefore keeps its full size and counts border pixels more than once.

2.1 Input and Output Format

```
srn ./hw1-1 path/to/input.png path/to/output.png
```

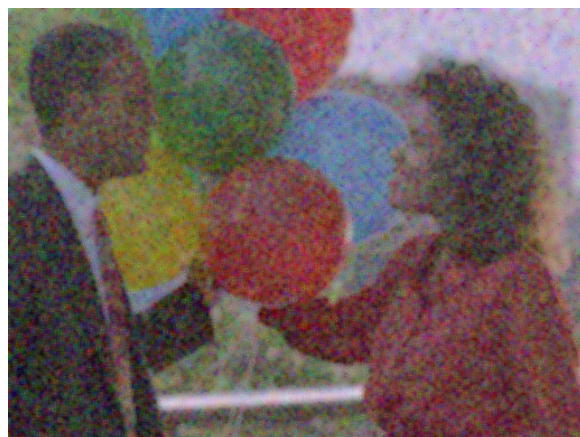
- **Input:** an RGB PNG image file containing a noisy image, given as `argv[1]`. Example: Figure 1a.
- **Output:** an RGB PNG image file containing the denoised image, written to the path given as `argv[2]`. Example: Figure 1b.

For example:

```
srn ./hw1-1 cases/hw1-1/t01.in.png output.png
```



(a) Example input.



(b) Example output.

2.2 Time Limit

Test images range from 640×480 to about 44 megapixels, with no fixed aspect ratio. Hidden test cases may be larger than the public ones; their time limits are set roughly in proportion to the running time of the sequential program on the same image. All public cases run with 8 cores. Some hidden cases run with 2 or 4 cores, with time limits adjusted accordingly.

Test case	Time limit
t01, t02	0.5 s
t03, t04	5.0 s
t05–t08	10.0 s

2.3 Correctness

The judge compares your output image pixel by pixel against the reference output, and accepts a test case when at least **99%** of pixels match exactly.

You can run the same comparison yourself with the `hw1-1-check` program:

```
hw1-1-check cases/hw1-1/t01.out.png output.png
```

It prints the percentage of matching pixels and exits with a non-zero status when that percentage is below 99%. The judge uses the same program, so a test case passes the judge if and only if it passes `hw1-1-check`.

3 HW 1-2: Feature Detection & Matching

HW 1-2 implements a SIFT-style feature pipeline¹ between two input images:

- **Scale space.** In each of four octaves, the program blurs the image at five sigmas, $1.6 \cdot 2^{s/3}$, with a separable Gaussian and subtracts adjacent blurs into difference-of-Gaussian (DoG) images. Each octave starts from the previous one downsampled by two.
- **Detection.** A DoG pixel becomes a keypoint when it is above or below all 26 of its neighbors in space and scale, its magnitude is at least 0.03, and it passes Lowe’s edge test with $r = 10$.
- **Orientation.** Each keypoint takes the peak of a 36-bin histogram of gradient directions around it, weighted by gradient magnitude and a Gaussian of width 1.5σ .
- **Descriptor.** The program rotates a 16×16 window to that orientation and sums gradient magnitudes into 8 direction bins in each of 4×4 cells. It normalizes the 128 sums, clips them at 0.2, and normalizes again.
- **Matching.** For each keypoint of the first image, the program finds the two nearest descriptors in the second image by Euclidean distance and keeps the pair when the nearest is under 0.75 times the second.

3.1 Input and Output Format

- **Input:** two RGB PNG images, given as `argv[1]` and `argv[2]`.
- **Output:** a plaintext file, written to the path given as `argv[3]`, listing the keypoints detected in each image and the matches found between them:

```
KEYPOINTS_A <n>
x y scale orientation
... (n lines)
KEYPOINTS_B <m>
```

¹D. G. Lowe, “Distinctive Image Features from Scale-Invariant Keypoints,” *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

```
x y scale orientation
... (m lines)
MATCHES <k>
idxA idxB distance
... (k lines)
```

`x`, `y` are keypoint coordinates in the original image's pixel space; `scale` and `orientation` (radians) describe the keypoint's local scale and dominant gradient direction. Each match line references a keypoint index into the preceding `KEYPOINTS_A` / `KEYPOINTS_B` list, plus the descriptor distance between them.

```
srunch ./hw1-2 path/to/imageA.png path/to/imageB.png path/to/output.txt
```

3.2 Time Limit

Test images range from 480×360 to 2640×1980 pixels. The two images of a case have the same size, except in `a06` and `b05`, where image B is half the size of image A.

Test case		Time limit
a01–a04	b01–b04	0.5 s
a05, a06	b05, b06	1.0 s
a07, a08	b07, b08	2.0 s

Hidden test cases may be larger than the public ones. All public cases run with 8 cores; some hidden cases run with 4 cores, with time limits adjusted accordingly.

3.3 Correctness

The judge compares your output against the output of the reference `hw1-2.cpp` within a numeric tolerance, because parallelizing the detection, descriptor and matching stages may reorder floating-point operations. Keypoints and matches are compared as unordered sets, and each of your keypoints is paired with the closest reference keypoint. There are two tolerance tiers:

- **Loose** (`txx`): position within $\pm 2\text{px}$, orientation within ± 0.1 rad, matches compared as unordered (`idxA`, `idxB`) pairs.
- **Tight** (`axx`, `bxx`): position within $\pm 0.5\text{px}$, orientation within ± 0.02 rad.

You can run the same comparison yourself with the `hw1-2-check` program, which lists every keypoint or match outside the tolerance:

```
hw1-2-check cases/hw1-2/a01.out.txt output.txt
hw1-2-check cases/hw1-2/a01.out.txt output.txt --tier loose
```

If the outputs match, it prints nothing and exits with status zero; otherwise it prints the differences and exits with a non-zero status. It checks the tight tier by default; pass `--tier loose` for the loose tier. The judge uses the same program, so a test case passes the judge if and only if it passes `hw1-2-check`.

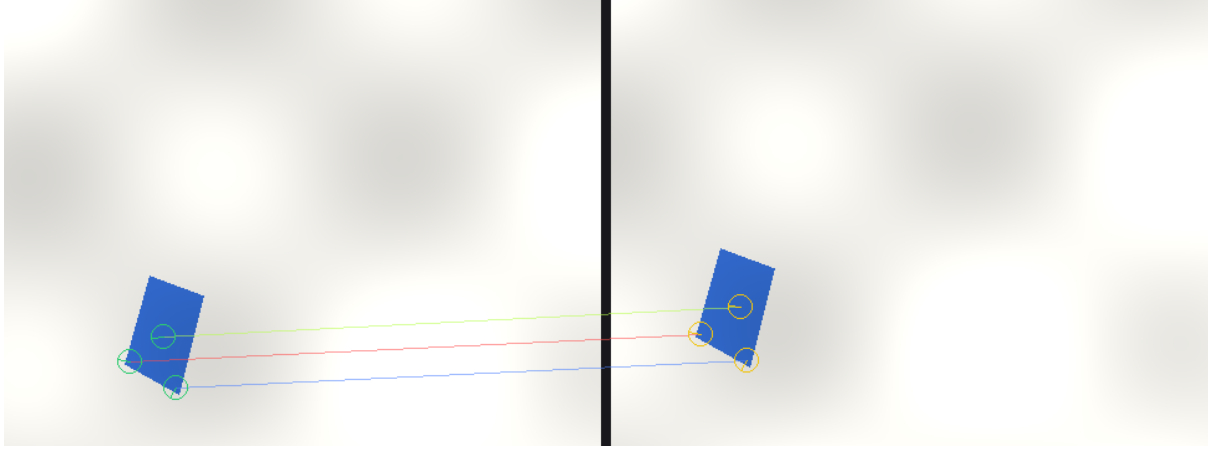


Figure 2: Reference output for public case `a01`, drawn by `hw1-2-viz`. Image A is on the left and image B on the right. Each keypoint is drawn as a circle whose radius is its scale, with a line segment along its orientation; each match is a line connecting the two images. Case `a01` contains a single shape with three keypoints, and image B is a shifted copy of image A, so the three match lines are parallel.

3.4 Seeing Your Output

The `hw1-2-viz` program draws an output file over the two input images it was computed from (Figure 2) and saves the result to `out/<case>.viz.png` :

```
hw1-2-viz a01.out.txt
hw1-2-viz output.txt --case a01
```

It infers the input images from the name of the output file; pass `--case` when the file is named differently. Comparing the visualization of your output with that of the reference output is a convenient way to interpret the differences reported by `hw1-2-check`. `hw1-2-viz` is a debugging aid only and plays no part in grading.

4 HW 1-3: Heat Diffusion

The sequential program (`hw1-3.cpp`) simulates heat diffusion through a cube of material with spatially varying conductivity, containing up to four spherical inclusions of a reactive material. The cube consists of $N \times N \times N$ cells. Each cell p has an initial temperature u_p and a conductivity a_p . In one time step, heat flows across each of the six faces of a cell in proportion to the temperature difference and the mean conductivity of the two cells:

$$r_p = u_p + \frac{1}{12} \sum_{q \sim p} (a_p + a_q) (u_q - u_p),$$

where q ranges over the six neighbors of p . Cells outside the cube are treated as having $u = 0$ and $a = 0$, so heat escapes through the surface. For an ordinary cell, r_p is the new temperature. A cell inside an inclusion additionally loses heat to a reaction whose rate increases with temperature; its new temperature u'_p is the result of `SUBSTEPS` sub-steps of

$$x_{m+1} + R(e^{x_{m+1}} - 1) = x_m, \quad x_0 = r_p,$$

each solved for x_{m+1} with exactly `NEWTON` iterations of Newton's method starting from x_m . R , `SUBSTEPS` and `NEWTON` are constants defined in the code. The reference output is defined

by this exact number of iterations rather than by the converged root, so changing the iteration count changes the answer. The program runs at most T steps. After each step it computes the total energy $\sum_p u_p^2$ of the cube, and it stops early once the energy has dropped to θ times its initial value or below.

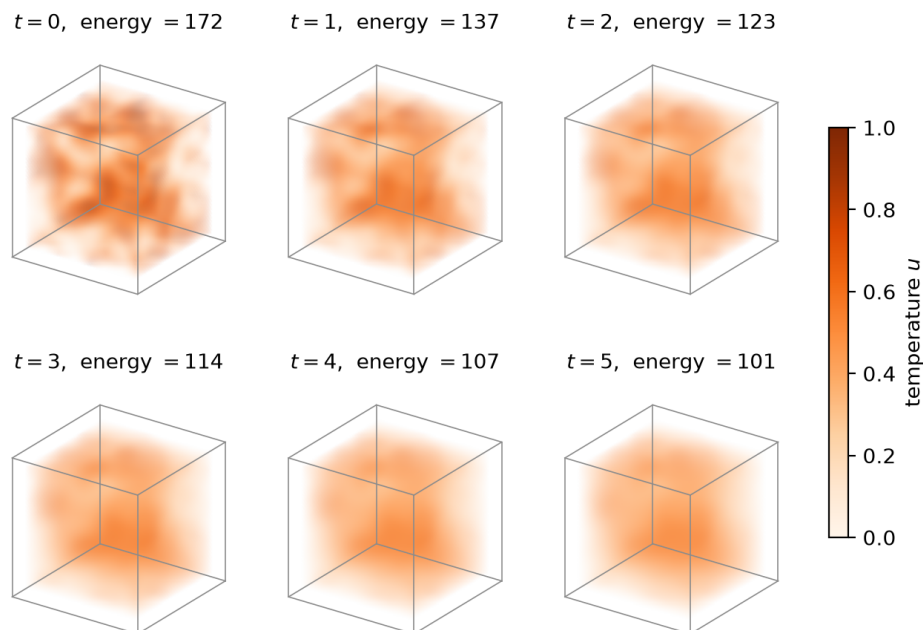


Figure 3: An $8 \times 8 \times 8$ cube with no inclusions over its first five steps. The random initial field smooths out within one or two steps, and the faces cool first as heat escapes through the surface; the total energy decreases accordingly.

Figure 4 illustrates the effect of inclusions. The initial temperature field, the conductivity field and the inclusions are all generated inside the program from the seed, which determines the number of inclusions (zero to four) and the position and radius of each. Do not change the generation method or the reaction; both are marked **DO NOT CHANGE** in the code. You may change everything else, including the array layout, the structure of the generation loop, and how and when the reactive cells are located and updated.

4.1 Input and Output Format

```
srunk ./hw1-3 N T seed theta path/to/output.txt
```

- **Input:** the cube's side N ($32 \leq N \leq 960$), the maximum number of time steps T ($0 \leq T \leq 1000$), the seed, and the stopping fraction θ ($0 \leq \theta < 1$; 0 means the program always runs all T steps), as `argv[1]` to `argv[4]`.
- **Output:** a text file written to the path given as `argv[5]`.

```
N steps
energy
t_1
t_2
...
```

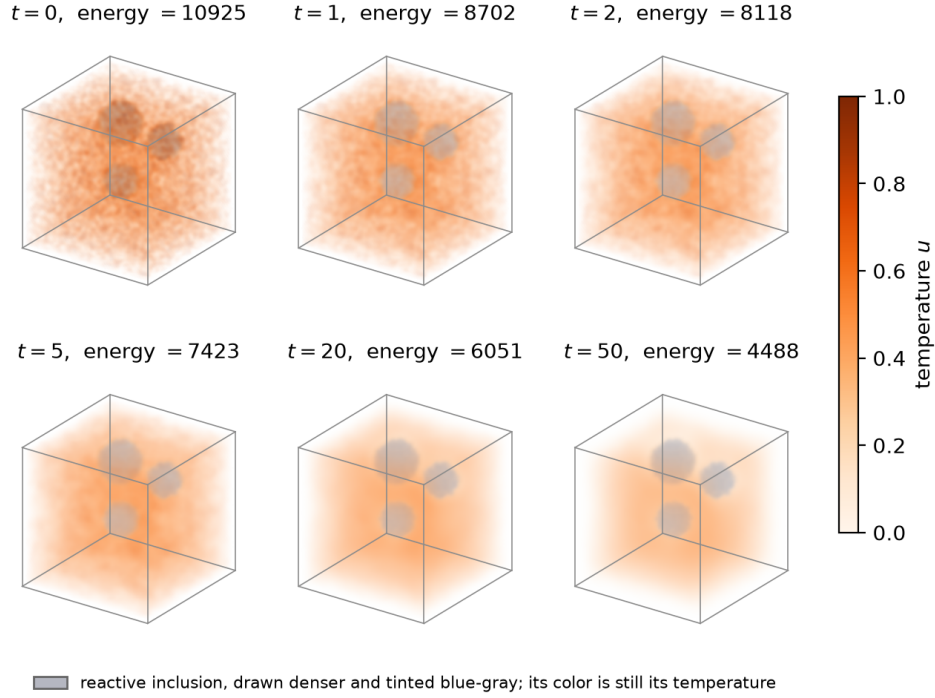


Figure 4: A $32 \times 32 \times 32$ cube with three inclusions, spheres of radius 4 to 5 cells drawn as denser blue-gray bodies; color still indicates temperature. The reaction reduces each sphere to about a tenth of its heat in the first step and to almost nothing by the fifth. The total energy falls by about 20% in the first step and by a few percent per step afterwards. The inclusions then act as heat sinks, like the surface: neighboring cells cool first, and the pale shells around the spheres at $t = 20$ and $t = 50$ show heat diffusing inward to be consumed.

`N` is the input N , and `steps` is the number of steps taken, which equals T unless the energy threshold stopped the simulation earlier. `energy` is the total energy $\sum_p u_p^2$ after those steps. `t_1`, `t_2`, ... are the temperatures of the $32 \times 32 \times 32$ sample cells, one per line, in the order the sequential program writes them. Print the temperatures with `%.17g`.

For example:

```
srunk ./hw1-3 512 8 1035 0 output.txt
```

runs public case p06. The parameters of each public case, together with its time limit and core count, are listed in `cases/hw1-3/pNN.txt`, and the expected output is in the corresponding `.out` file.

4.2 Time & Memory Limit

Each case runs with a **16 GB** memory limit. The provided sequential program **exceeds this limit** for large N , so you must reduce its memory footprint to pass those cases.

Test case	Time limit	Cores
p01, p02	1.0 s	2, 8
p03	2.0 s	8
p04	5.0 s	2
p06	5.0 s	4
p05	5.0 s	8
p07, p08, p09	7.5 s	8
p10	10.0 s	8

Hidden test cases may be larger than the public ones and use any combination of the core counts and time limits above.

4.3 Correctness

The judge compares your output against the reference output. N and `steps` must match exactly. All other values are compared with a relative tolerance: your value x matches the reference value y when

$$|x - y| \leq \varepsilon \cdot \max(|x|, |y|),$$

with $\varepsilon = 10^{-7}$ for the energy and $\varepsilon = 10^{-9}$ for each of the 32^3 sample temperatures. The energy threshold of each case is chosen with a margin, so that rounding differences between correct programs (for example, from summing in a different order) cannot change the step at which the simulation stops.

You can run the same comparison yourself with the `hw1-3-check` program:

```
hw1-3-check cases/hw1-3/p05.out output.txt
```

It reports any mismatch in the header or the energy, and the lattice position of the first differing sample, and exits with a non-zero status if anything differs. The judge uses the same program, so a test case passes the judge if and only if it passes `hw1-3-check`.

5 Submission

Please zip the following files into a single archived file named `<studentID>.tar` :

- `hw1-1.cpp` — the source code of your HW 1-1 implementation. Required.
- `hw1-2.cpp` — the source code of your HW 1-2 implementation. Include this only if you attempted HW 1-2.
- `hw1-3.cpp` — the source code of your HW 1-3 implementation. Include this only if you attempted HW 1-3.
- `Makefile` — optional. Submit this file if you want to change the build command.

The first character of your student ID in the filename **MUST** be lowercase, and the archive must contain a directory of the same (lowercase) name rather than the bare files. Submissions that do not follow this format lose points.

For example:

```
mkdir <studentID>
cp <your_file_A> <studentID>/
```

```
cp <your_file_B> <studentID>/  
...  
tar cvf <studentID>.tar <studentID>
```

Then submit the file `<studentID>.tar` .

Use exactly the file names listed above. Incorrect names such as `hw1.c` , `HW1.cc` or `hw1-1.cc` lose points.

6 Grading

The three problems are scored independently and the scores are added. Your score on one problem does not affect your score on another, so attempting a problem can never lower your total.

Problem	Total	Correctness	Performance
HW 1-1	30%	25	5
HW 1-2	40%	35	5
HW 1-3	30%	10	20
	100%	70	30

Correctness for a problem is $\frac{X}{N}$ of its correctness points, where X is the number of test cases you pass and N is the total number of test cases for that problem, **public and hidden**. The public judge (Section 7.2) runs only the public cases.

Performance for a problem is based on your total execution time over all of its test cases, public and hidden, ranked against the other students who submitted that problem. A failed test case counts as a fixed penalty of **30 seconds** instead of its running time. Performance is scored relative to the class rather than against a fixed time, so that you do not need to optimize parts of the program that are irrelevant to this course (for example, file output).

7 Appendix

The public test cases and the provided programs may contain bugs. If you suspect one, ask on NTU COOL.

7.1 Public Test Cases

The public test cases are in `/home/share/pp26/cases/{hw1-1,hw1-2,hw1-3}` . They are large and read-only, so we recommend linking to the directory rather than copying it into your home directory:

```
ln -s /home/share/pp26/cases ~/hw1/cases
```

7.2 Automatic Judge

The commands `hw1-1-judge` , `hw1-2-judge` and `hw1-3-judge` test your code on all public test cases of the corresponding problem and submit your execution times to that problem's scoreboard, where you can compare your performance with the rest of the class:

- <http://140.112.91.83/leaderboard/hw1-1>
- <http://140.112.91.83/leaderboard/hw1-2>
- <http://140.112.91.83/leaderboard/hw1-3>

The judge also prints the scoreboard link when it finishes.

Run the judge in the directory containing your source files (`hw1-1.cpp` , `hw1-2.cpp` and/or `hw1-3.cpp`). It builds with your `Makefile` if one is present, and with the provided `Makefile` otherwise.

The automatic judge enforces a 10-minute cooldown between runs, to prevent flooding the cluster with jobs.

You can also run the judge on a single test case, for example `hw1-1-judge t01` or `hw1-2-judge a01` . Single-case runs are exempt from the cooldown but do not update the scoreboard.

```
o dogeon@seiren ~/lab1 on ʒ main [!]  
•> lab1-judge  
building: make lab1  
mpicxx -std=c++11 -O3 -o lab1 lab1.cpp  
  
judging 3 case(s)  
TC  STAT  NEW  BEST  
n1  AC     0.24  -  ↓  
n2  AC     0.26  -  ↓  
n4  AC     0.27  -  ↓  
  
Total:3/3  0.77 30.00 ↓  
  
leaderboard: http://140.112.91.83/leaderboard/lab1
```

Figure 5: Example judge results.

The judge commands and the scoreboards are provided for your convenience and are not used for grading. Your grade is based on the code you submit to NTU COOL, evaluated on both the public and the hidden test cases.

7.3 Judge Verdict Table

The judge reports one of the following verdicts for each test case:

Code	Verdict	Explanation
IE	internal error	a bug in the judge; please report it to the TAs
TLE+	time limit exceed+	execution time > time limit + 10 seconds
TLE	time limit exceed	execution time > time limit
RE	runtime error	your program did not return 0 or was terminated by a signal
CE	compile error	your program did not compile successfully
NO	no output	your program did not produce an output file
WA	wrong answer	your output is incorrect
AC	accepted	you passed the test case

Table 1: Judge verdicts.